End to End PostgreSQL Portfolio Project

Online Book Store Database

Project Description:

This SQL project demonstrates the design and querying of an Online Book Store database using PostgreSQL. It includes database and table creation, table relationships, data import from CSV files, basic SQL queries, and advanced analytical queries using JOIN, GROUP BY, HAVING, aggregate functions, and filtering conditions,etc.

SQL Implementation:

```
--DATABASE CREATION
CREATE DATABASE OnlineBookStore;

--Switch to the Database
\c OnlineBookStore;

-- TABLE CREATION
--1)Books Table
DROP TABLE IF EXISTS Books;
CREATE TABLE Books (
        Book_ID SERIAL PRIMARY KEY,
        Title     VARCHAR(100),
        Author VARCHAR(100),
        Genre   VARCHAR(100),
        Published_Year INT,
        Price    NUMERIC(10,2),
        Stock   INT
);

--2)Costumers Table
DROP TABLE IF EXISTS Customers;
CREATE TABLE Customers (
        Customer_ID  SERIAL PRIMARY KEY,
        Name   VARCHAR(100),
        Email   VARCHAR(100),
        Phone  INT,
        City     VARCHAR(100),
        Country VARCHAR(100)
);

--3)Orders Table
DROP TABLE IF EXISTS Orders;
CREATE TABLE Orders (
        Order_ID SERIAL PRIMARY KEY,
        Customer_ID  INT REFERENCES Customers(Customer_ID),
```

```
                Book_ID INT  REFERENCES Books(Book_ID),
                Order_Date DATE,
                Quantity      INT,
                Total_Amount NUMERIC(10,2)
);

SELECT * FROM Books;
SELECT * FROM Customers;
SELECT * FROM Orders;
```

-- DATA IMPORT

--Import Data Into Books Table

```
COPY Books(Book_ID, Title, Author, Genre, Published_Year, Price, Stock)
FROM 'Books.csv'
CSV HEADER;
```

--Import Data Into Costumer Table

```
COPY Customers(Customer_ID, Name, Email, Phone, City, Country)
FROM 'Customers.csv'
CSV HEADER;
```

--Import Data Into Orders Table

```
COPY Orders(Order_ID, Customer_ID, Book_ID, Order_Date, Quantity, Total_Amount)
FROM 'Orders.csv'
CSV HEADER;
```

-- BASIC SQL QUERIES

-- 1) Retrieve all books in the "Fiction" genre:

```
SELECT * FROM Books
WHERE Genre='Fiction';
```

-- 2) Find books published after the year 1950:

```
SELECT * FROM Books
WHERE published_year>1950 ORDER BY published_year ASC;
```

-- 3) List all customers from the Canada:

```
SELECT * FROM Customers
WHERE country='Canada';
```

-- 4) Show orders placed in November 2023:

```
SELECT * FROM Orders
WHERE order_date BETWEEN'2023-Nov-01' AND '2023-Nov-30';
```

-- 5) Retrieve the total stock of books available:

```
SELECT SUM(stock) AS Total_stock_available_of_books FROM Books;
```

-- 6) Find the details of the most expensive book:
```
SELECT title ,price FROM Books
ORDER BY price DESC
LIMIT 3;
```

-- 7) Show all customers who ordered more than 1 quantity of a book:
```
SELECT * FROM Orders
WHERE quantity>1 ORDER BY quantity ASC ;
```

-- 8) Retrieve all orders where the total amount exceeds $20:
```
SELECT * FROM Orders
WHERE total_amount>20  ORDER BY total_amount ASC;
```

-- 9) List all genres available in the Books table:
```
SELECT DISTINCT genre FROM Books;
```

-- 10) Find the book with the lowest stock:
```
SELECT * FROM Books ORDER BY stock
LIMIT 10;
```

-- 11) Calculate the total revenue generated from all orders
```
SELECT SUM(total_amount) AS total_revenue_generated_of_all_books FROM Orders;
```

-- ADVANCED SQL QUERIES

-- 1) Retrieve the total number of books sold for each genre:
```
SELECT b.genre ,SUM(o.Quantity) AS Total_Books_Sold
FROM Orders o
JOIN Books b ON o.book_id=b.book_id
GROUP BY b.genre;
```

-- 2) Find the average price of books in the "Fantasy" genre:
```
SELECT AVG(price) AS Fantasy_Genre_Average_Price FROM Books
WHERE genre='Fantasy' ;
```

-- 3) List customers who have placed at least 2 orders:
```
SELECT c.customer_id ,c.name ,COUNT(o.order_id) AS Order_Counts
FROM Orders o
JOIN Customers c ON c.customer_id=o.customer_id
GROUP BY c.customer_id, c.name
HAVING COUNT(o.order_id) >=2;
```

-- 4) Find the most frequently ordered book:
```
SELECT o.Book_id, b.title, COUNT(o.order_id) AS ORDER_COUNT
FROM orders o
JOIN books b ON o.book_id=b.book_id
GROUP BY o.book_id, b.title
```

```
ORDER BY ORDER_COUNT DESC LIMIT 1;
```

-- 5) Show the top 3 most expensive books of 'Fantasy' Genre :

```
SELECT * FROM books
WHERE genre ='Fantasy'
ORDER BY price DESC LIMIT 3;
```

-- 6) Retrieve the total quantity of books sold by each author:

```
SELECT b.author, SUM(o.quantity) AS Total_Books_Sold
FROM orders o
JOIN books b ON o.book_id=b.book_id
GROUP BY b.Author;
```

-- 7) List the cities where customers who spent over $30 are located:

```
SELECT DISTINCT c.city, total_amount
FROM orders o
JOIN customers c ON o.customer_id=c.customer_id
WHERE o.total_amount > 30;
```

-- 8) Find the customer who spent the most on orders:

```
SELECT c.name ,o.customer_id,SUM(o.total_amount) AS Total_Spent
FROM Orders o
JOIN Customers c ON o.customer_id=c.customer_id
GROUP BY c.name ,o.customer_id
ORDER BY Total_Spent DESC LIMIT 3;
```

--9) Calculate the stock remaining after fulfilling all orders:

```
SELECT b.book_id, b.title, b.stock, COALESCE(SUM(o.quantity),0) AS
Orders_Quantity,
b.stock-COALESCE(SUM(o.quantity),0) AS Remaining_Quantity
FROM Books b
LEFT JOIN Orders o ON b.book_id=o.book_id
GROUP BY b.book_id
ORDER BY b.book_id;
```